

Optimized Kernel Fusion for Accelerating Multimodal Inference on Edge Device

트랙: 연구

팀명: Nitro

팀원: 2217011 박세진, 2271029 박윤형, 2376004 강선화

지도교수: 윤명국

Table of contents

I. Team Info.

- A. Project Title
- B. Team Information
- C. Team Members

II. Project Summary

- A. Problem Definition
- B. Comparison with Existing Research
- C. Proposed Methodology
- D. Expected Effects and Significance
- E. Key Features

III. Project Design

- A. Requirements Definition
- B. Overall System Architecture
- C. Design of Major Engines and Features
- D. Implementation of Major Features
- E. Additional Notes

I. Team Information

A. Project Title

Optimized Kernel Fusion for Accelerating Multimodal Inference on Edge Device

B. Team Information

29번 팀 Nitro

C. Team Members

- 강선화(2376004):
 - Multimodal Model 구조 분석 및 프로파일링
 - Kernel Fusion 구현
 - 성능 비교 실험 진행
 - 문서 정리 및 회의록 관리
- 박세진(2217011):
 - 프로젝트 전체 연구 방향 설계
 - Multimodal Model 구조 분석 및 프로파일링
 - Kernel Fusion 구현
 - 성능 비교 실험 진행
 - 문서 정리 및 회의록 관리
- 박윤형(2271029):
 - Multimodal Model 구조 분석 및 프로파일링
 - Kernel Fusion 구현
 - 성능 비교 실험 진행
 - 문서 정리 및 회의록 관리

II. Project Summary

A. Project Definition

최근 멀티모달 대형 모델(Large Multimodal Models, LMMs)은 텍스트와 이미지 등 다양한 모달리티를 통합적으로 처리하기 위해 Transformer 기반 구조를 채택하고 있으며, 성능 향상을 위해 모델 규모, 입력 해상도, 그리고 visual token 수가 지속적으로 증가하고 있다. 이러한 변화는 연산량과 메모리 요구량을 크게 증가시키며, 특히 attention 및 feed-forward network(FFN)에서 수행되는 대규모 행렬 연산(GEMM)의 비중을 확대시키고 있다.

Attention 메커니즘은 Query, Key, Value(QKV) 간의 유사도를 기반으로 입력 요소 간의 중요도를 계산하는 연산으로, 대부분 matrix multiplication과 softmax 연산으로 구성된다. 이로 인해 Transformer 기반 모델에서는 GEMM과 attention 연산이 전체 계산의 대부분을 차지한다.

그러나 엣지 디바이스 환경에서는 이러한 연산 구조가 비효율적으로 실행된다. Embedded GPU는 제한된 연산 자원과 메모리 대역폭을 가지며, 여러 kernel을 완전히 병렬적으로 실행하는 데 제약이 존재한다. 또한 multistream execution 환경에서도 kernel 간 완전한 병렬성이 보장되지 않아 context switching overhead가 발생하며, 이는 전체 latency 증가로 이어진다.

Transformer block 내부에서는 GEMM과 같은 대규모 연산뿐 아니라, QKV projection 이후의 split, reshape, transpose, 그리고 bias addition, activation, layer normalization과 같은 연산이 각각 독립적인 CUDA kernel로 실행된다. 이로 인해 하나의 Transformer block 실행 과정에서 수십 개 이상의 kernel이 순차적으로 호출되며, kernel launch overhead와 global memory access 비용이 누적된다.

특히 QKV split 및 transpose 연산은 계산량은 거의 없고 메모리 접근이 대부분을 차지하는 memory-bound kernel로, 전체 attention pipeline에서 의미 있는 latency를 차지하는 병목으로 나타난다. 이러한 특성은 Nsight Systems 및 Nsight Compute의 roofline 분석을 통해 확인되며, 연산 대비 메모리 접근 비율이 높은 비효율적인 실행 패턴을 보인다.

따라서 본 연구에서는 이러한 병목을 해결하기 위해 Transformer 내부 연산의 kernel 실행 구조를 재구성하고, kernel launch overhead 및 memory traffic을 동시에 감소시키는 접근이 필요하다고 판단하였다.

B. Comparison with Existing Research

GPU inference 최적화를 위한 대표적인 방법으로는 TensorRT와 CUTLASS 기반 kernel 최적화 기법이 존재한다.

TensorRT는 연산 그래프 수준에서 사전에 정의된 operator pattern을 기반으로 kernel fusion을 수행하는 rule-based heuristic 접근을 사용한다. 이 방식은 CNN 및 표준 Transformer workload에 대해서는 효과적이지만, attention preparation, QKV transformation, epilogue computation과 같은 비정형적인 연산 흐름에서는 fusion 기회를 충분히 활용하지 못하는 한계를 가진다. 특히 일부 Transformer 연산은 여전히 다수의 kernel로 분리되어 실행되어 kernel launch overhead를 유발한다.

CUTLASS는 텐서 코어 기반의 고성능 GEMM 연산을 제공하며, Efficient GEMM Template(EVT)를 통해 epilogue 단계에서 element-wise 연산을 fusion할 수 있다. 그러나 이 접근은 activation, bias addition 등 단순 연산에 주로 적용되며, QKV split, reshape, transpose와 같은 memory-bound tensor transformation을 포함한 전체 연산 흐름까지 통합하기에는 제한적이다. 또한 kernel configuration 변경 시 재컴파일이 필요하며, 개발 복잡도가 증가하는 문제도 존재한다.

결과적으로 기존 방법들은 Transformer block 전반에서 발생하는 kernel launch overhead 및 memory-bound 연산을 구조적으로 줄이기에는 한계가 있으며, 특히 엣지 디바이스 환경에서의 비효율을 충분히 해결하지 못한다.

C. Proposed Methodology

본 연구에서는 멀티모달 모델의 Transformer block에서 발생하는 성능 병목을 정량적으로 분석하고, 이를 기반으로 kernel-level 최적화를 수행하는 접근을 제안한다.

먼저, Nsight Systems를 활용하여 Transformer block 내부에서 실행되는 CUDA kernel들의 호출 패턴을 분석하였다. 분석 결과, 하나의 Transformer block 실행 과정에서 GEMM과 같은 대규모 연산뿐 아니라, QKV projection 이후의 split, transpose, reshape, 그리고 bias addition, activation, layer normalization과 같은 다수의 연산이 각각 독립적인 kernel로 수행되며, 수십 개 이상의 kernel이 순차적으로 호출되는 구조를 확인하였다. 이러한 구조는 kernel launch overhead의 누적과 함께 global memory access 증가로 이어져 전체 latency 증가의 주요 원인으로 작용한다.

이후 Nsight Compute를 이용한 roofline 분석을 통해 각 kernel의 성능 특성을 평가하였다. 그 결과, QKV split 및 transpose 연산은 낮은 arithmetic intensity를 가지며 memory bandwidth에 의해 성능이 제한되는 대표적인 memory-bound kernel로 나타났다. 또한 해당 연산은 계산량은 작지만 전체 attention pipeline에서 의미 있는 latency 비중을 차지하는 것으로 확인되었다. 이러한 분석을 바탕으로, 본 연구에서는 주요 병목 구간을 다음 두 가지로 정의하였다:

1) QKV Split + Transpose Fusion

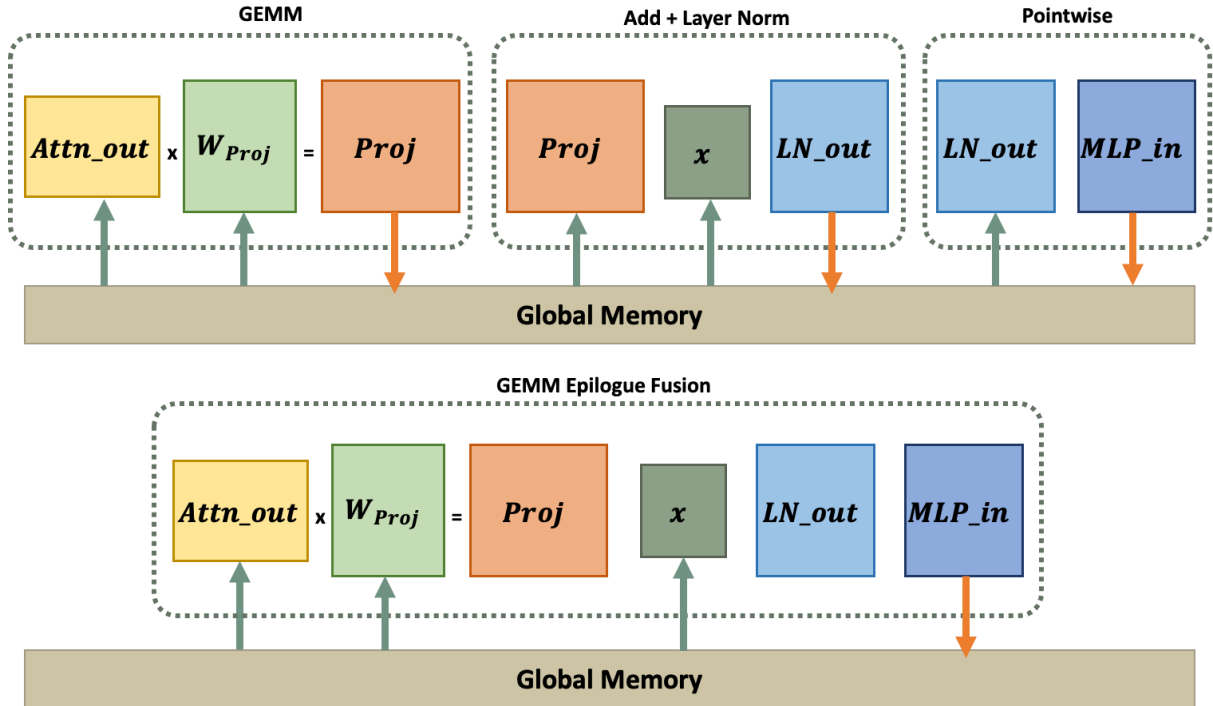


그림 1. QKV Split & Transpose 커널 퓨전.

QKV split 및 transpose 연산은 연산량 대비 메모리 접근 비중이 높은 memory-bound kernel로, 반복적인 global memory read/write를 유발하며 latency에 비효율적으로 기여한다. Profiling 결과, 해당 kernel은 전체 attention 연산 과정에서 무시할 수 없는 latency 비중을 차지하며, 특히 엡지 디바이스 환경에서 그 영향이 더욱 크게 나타난다.

이를 해결하기 위해, 본 연구에서는 QKV projection GEMM의 output 단계에서 split 및 transpose 연산을 동시에 수행하도록 kernel을 재구성하는 fusion 전략을 제안한다. 이를 통해 기존에 별도의 kernel로 수행되던 tensor transformation 연산을 제거하고, intermediate tensor의 global memory 접근을 최소화한다.

이러한 접근은 memory-bound kernel의 실행을 제거함으로써 memory traffic을 감소시키고, kernel launch 횟수를 줄여 전체 latency 개선으로 이어질 것으로 기대된다. 또한 해당 기법은 서버 GPU뿐 아니라 엡지 디바이스 환경에서도 적용 가능성을 검증하기 위해 다양한 환경에서 성능을 측정할 예정이다.

2) GEMM Epilogue Fusion (Residual + Bias + Activation + LayerNorm)

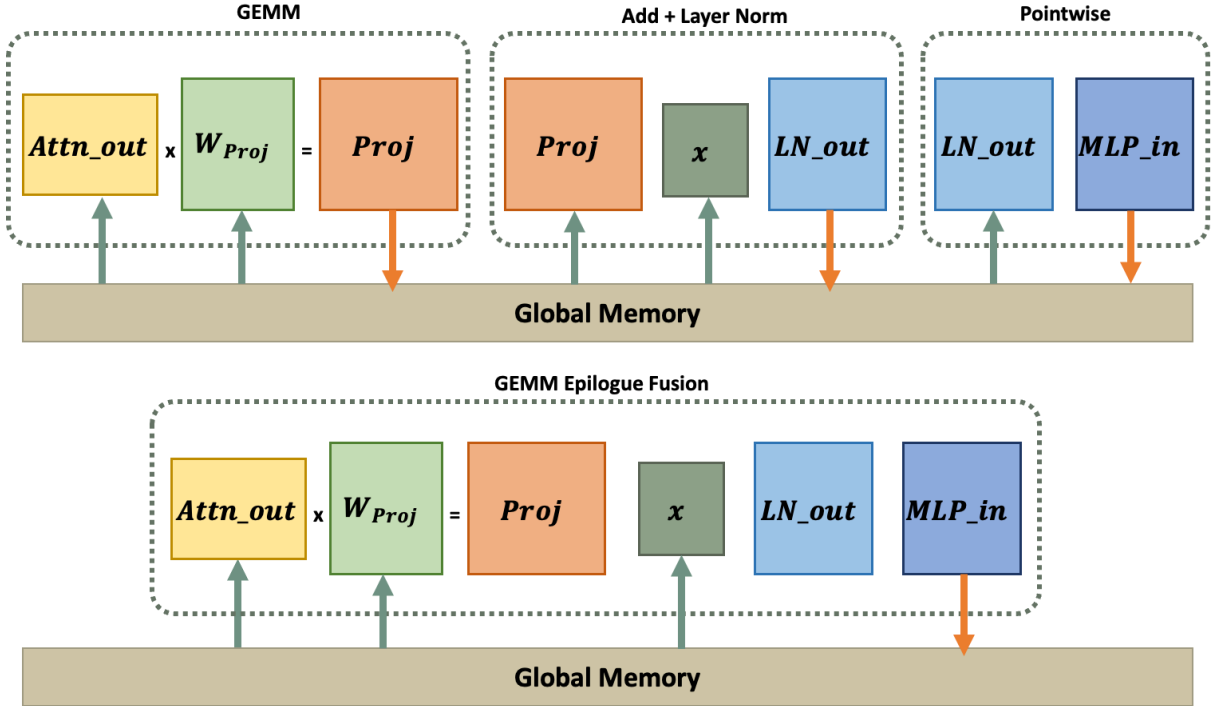


그림 2. GEMM 에필로그 퓨전.

Transformer block에서는 GEMM 이후 residual addition, bias addition, activation, layer normalization과 같은 연산이 연속적으로 수행되며, 각각이 별도의 kernel로 실행된다. 이러한 구조는 intermediate tensor를 global memory에 반복적으로 저장하고 다시 읽어오는 과정을 발생시켜 memory access overhead를 증가시킨다. 본 연구에서는 이러한 연산들을 GEMM kernel의 epilogue 단계에 통합하여, 하나의 fused kernel에서 연속적으로 수행하는 방식을 제안한다. 이를 통해 kernel 호출 횟수를 감소시키고, intermediate tensor의 불필요한 memory access를 제거할 수 있다.

특히 layer normalization은 reduction 연산을 포함하기 때문에 단순한 element-wise 연산과 달리 thread 간 synchronization과 tile-based parallelization이 요구되며, 이에 따른 kernel 설계의 복잡성이 존재한다. 따라서 해당 fusion은 단순 연산 결합을 넘어, 효율적인 병렬 실행 구조를 고려한 커널 설계가 필요하다.

현재 GEMM epilogue fusion은 구현 및 최적화가 진행 중이며, 향후 profiling 기반 성능 분석을 통해 그 효과를 검증할 예정이다.

종합적으로, 본 연구는 Nsight 기반 profiling을 통해 Transformer block 내부의 kernel execution 패턴과 memory-bound 병목을 식별하고, 이를 해결하기 위한 QKV fusion과 GEMM epilogue fusion의 두 가지 kernel-level 최적화 전략을 제안한다. 제안된 kernel fusion은 CUTLASS 기반 GEMM 구조 및 epilogue 확장 메커니즘을 활용하여 구현되며, 기존 연산 흐름을 유지하면서 kernel-level에서의 연산 통합을

수행한다. 이러한 접근을 통해 kernel launch overhead와 memory traffic을 동시에 감소시키고, 멀티모달 모델의 inference latency를 개선하는 것을 목표로 한다.

D. Expected Effects and Significance

본 연구를 통해 다음과 같은 효과를 기대할 수 있다.

- kernel fusion을 통해 kernel launch 횟수를 감소시켜 launch overhead를 줄일 수 있다.
- memory-bound kernel을 제거하고 intermediate tensor의 global memory 접근을 최소화하여 memory traffic을 감소시킬 수 있다.
- Transformer block 내부 연산의 실행 효율을 개선하여 전체 inference latency를 단축할 수 있다.
- 엣지 디바이스 환경에서 멀티모달 모델의 실시간 추론 가능성을 향상시킬 수 있다.

또한 본 연구는 Transformer 기반 연산 패턴을 중심으로 최적화를 수행함으로써, CLIP, BLIP-2, SigLIP-2, Qwen2.5-VL과 같이 서로 다른 구조의 멀티모달 모델에 공통적으로 적용 가능한 범용적 최적화 기법을 제안한다.

현재 QKV fusion에 대해서는 profiling 기반 분석과 구현을 통해 성능 개선 가능성을 확인하였으며, GEMM epilogue fusion은 구현 및 최적화가 진행 중이다.

E. Key Features

본 연구에서 구현 및 검증할 주요 기능은 다음과 같다.

- Nsight Systems를 활용한 Transformer block 내부 kernel execution 분석
- Nsight Compute roofline 기반 memory-bound kernel 식별
- QKV split 및 transpose kernel의 latency 비중 분석 및 최적화
- QKV fusion kernel 설계 및 구현, 성능 평가 (서버 및 엣지 환경 포함)
- GEMM epilogue fusion kernel 설계 및 구현 (residual, bias, activation, layer normalization 포함)
- TensorRT baseline과의 성능 비교
- 다양한 멀티모달 모델(CLIP, BLIP-2, SigLIP-2, Qwen2.5-VL)에 대한 적용 및 검증
- 성능 평가 지표 분석
 - inference latency
 - kernel launch overhead
 - memory traffic
 - accuracy 변화 여부

III. Project-Design

A. Requirements Definition

본 과제의 핵심 요구사항은 멀티모달 모델의 Transformer block 내부에서 발생하는 비효율적인 kernel execution 구조를 정량적으로 분석하고, 이를 개선하기 위한 kernel fusion 기법을 설계 및 검증하는 것이다. 최근 Transformer 기반 멀티모달 모델은 attention 및 FFN 중심의 대규모 연산 구조를 가지며, 특히 block 내부에서 GEMM 이후에 다수의 tensor transformation 및 element-wise 연산이 독립적인 CUDA kernel로 분리되어 수행된다. 이러한 구조는 kernel launch overhead를 증가시키고 intermediate tensor의 반복적인 global memory access를 유발하므로, 전체 추론 지연시간 증가의 주요 원인으로 작용한다. 따라서 본 연구에서는 Nsight 기반 profiling을 통해 병목 kernel을 식별하고, 이를 바탕으로 kernel-level fusion을 수행하여 memory traffic과 launch overhead를 동시에 감소시키는 것을 주요 요구사항으로 설정한다.

우선 Transformer block 내부의 kernel 호출 구조를 세밀하게 분석할 필요가 있다. Nsight Systems 분석 결과, Transformer block은 단일 대규모 연산으로 구성되지 않으며, `CLS token concat + position embedding add + layer normalization, QKV bias add/reshape, QKV projection, Q, K, V split + transpose, multi-head attention, attention output projection, residual add + layer normalization, MLP bias add/reshape, MLP FC1, output initialization, MLP FC2`와 같은 다수의 kernel이 순차적으로 호출되는 구조를 가진다. 즉, Tensor Core 기반 GEMM kernel뿐 아니라 reshape, transpose, add, normalization과 같은 보조 연산들이 block 내부에 빈번하게 삽입되어 있으며, 이들이 각각 독립 kernel로 수행됨으로써 연산 흐름이 과도하게 세분화되어 있다. 따라서 fusion 대상 연산을 도출하기 위해서는 Transformer block 내 kernel sequence를 분해하고, 각 kernel의 기능과 실행 시간 비중을 함께 파악해야 한다.

이와 함께, 연산 집약도는 낮고 메모리 접근 비중은 높은 memory-bound kernel을 식별하는 과정이 필요하다. profiling 결과, Transformer block 내의 모든 kernel이 동일한 병목 특성을 보이는 것은 아니며, 특히 `QKV bias add/reshape, Q, K, V split + transpose, Residual Add + LayerNorm, MLP bias add/reshape`와 같은 연산들은 높은 FLOPs를 요구하지 않음에도 불구하고 대량의 tensor read/write를 수반한다. 이러한 kernel들은 arithmetic intensity가 낮고, roofline 관점에서 memory bandwidth에 의해 성능이 제한되는 전형적인 memory-bound workload에 해당한다. 그중에서도 `Q, K, V split + transpose`는 attention preparation 단계에서 반복적으로 수행되며, intermediate tensor를 global memory에 저장한 후 다시 읽어오는 과정을 수반하므로 execution time과 DRAM access 측면에서 모두 비효율적인 병목으로 작용한다. 또한 residual add, bias add, layer normalization 역시 element-wise 연산과 reduction이 결합된 구조로 인해 연산 밀도는 낮은 반면 memory access 비중은 높다. 따라서 본 연구에서는 단순히 실행 시간이 긴 kernel만을 대상으로 삼는 것이 아니라, 실제 성능 저하를 유발하는 low-AI memory-bound kernel을 우선적인 최적화 대상으로 정의한다.

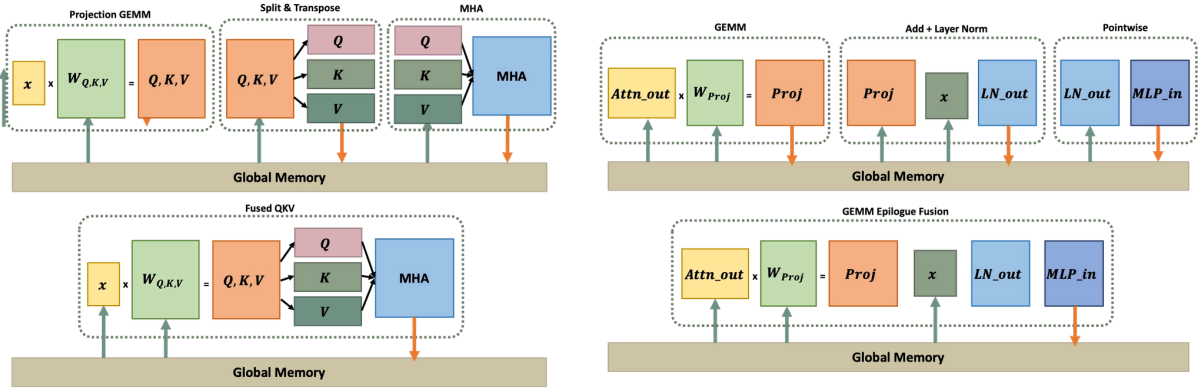


그림 3. (1) QKV Fusion, (2) GEMM+Residual Add+LN+Bias Add Fusion

이러한 분석을 바탕으로, 본 연구에서는 두 가지 kernel fusion 전략을 핵심 solution으로 설정한다. 첫 번째는 QKV Fusion으로, QKV projection GEMM 이후 별도의 kernel로 수행되던 split 및 transpose를 GEMM의 output 단계에 통합하는 방식이다. 기존의 `GEMM → global memory write → split/transpose kernel → global memory write → MHA input` 구조를 `GEMM + split/transpose fused kernel → MHA input` 구조로 재구성함으로써, intermediate tensor 저장 및 재로드를 최소화할 수 있다. 두 번째는 GEMM + Residual Add + LayerNorm + Bias Add Fusion으로, attention output projection 또는 MLP projection 이후 수행되던 residual addition, bias addition, layer normalization을 GEMM epilogue에 통합하는 방식이다. 이를 통해 intermediate tensor를 global memory에 반복적으로 기록하지 않고 연속적으로 처리할 수 있으므로, kernel launch 횟수와 DRAM traffic을 동시에 줄일 수 있다. 특히 layer normalization은 reduction 및 synchronization이 수반되므로, 단순한 element-wise fusion보다 높은 구현 복잡도를 가지며, tile-based parallelization과 numerically stable reduction을 고려한 설계가 요구된다.

또한 제안한 각 feature의 효과를 검증하기 위한 실험 체계 역시 요구사항에 포함된다. QKV Fusion의 경우 TensorRT baseline 대비 fused implementation의 효과를 정량적으로 분석하기 위해, 모델별 sequence length 변화에 따른 `GEMM 대비 split_transpose의 실행 시간 비율`, `GEMM 대비 split_transpose의 DRAM access 비율`, 그리고 최종 `speedup`을 측정해야 한다. 실험 대상은 encoder 계열과 decoder 계열을 모두 포함하도록 CLIP-base-text, BERT-base, BERT-large, OPT-2.7B, Llama-7B로 구성하며, sequence length 증가에 따라 split_transpose의 병목 정도가 어떻게 변화하는지를 확인한다. GEMM Epilogue Fusion에 대해서는 구현 완료 후 baseline과 비교하여 kernel launch count, DRAM transaction 수, arithmetic intensity, block latency, 그리고 정확도 변화를 함께 평가해야 한다. 본 연구의 목적은 단순한 속도 향상에 그치지 않고 실제 멀티모달 모델 적용 가능성을 입증하는 데 있으므로, fused kernel 적용 이후에도 output correctness가 유지되는지 반드시 확인해야 한다.

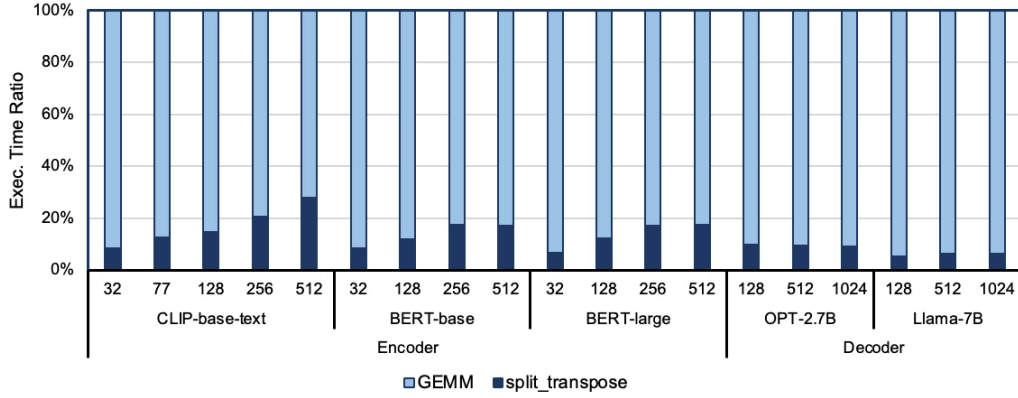


그림 4. GEMM & Split_Transpose Kernel Exec Time Ratio

QKV Fusion의 구현 결과는 이러한 요구사항의 타당성을 뒷받침한다. 먼저 execution time ratio 분석 결과, `split_transpose` kernel은 모든 모델에서 무시하기 어려운 실행 시간 비중을 차지하였으며, sequence length가 증가할수록 그 비중 또한 증가하는 경향을 보였다. 특히 encoder 계열에서는 긴 sequence에서 `split_transpose`의 비중이 더욱 뚜렷하게 증가하였고, CLIP-base-text의 경우 sequence length 512에서 약 30%에 근접하는 수준까지 상승하였다. 이는 split/transpose가 단순한 보조 연산이 아니라, 긴 입력 조건에서 attention preparation 단계의 실질적인 병목이 될 수 있음을 의미한다. 반면 decoder 계열에서는 GEMM의 계산 비중이 상대적으로 크게 유지되었으나, 그림에도 `split_transpose`가 일정 수준의 latency overhead를 지속적으로 유발하는 것으로 나타났다.

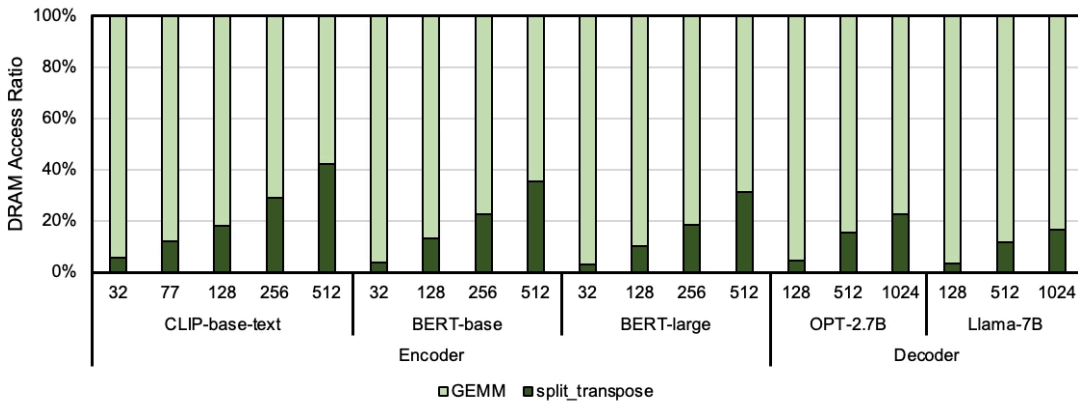


그림 5. GEMM & Split_Transpose DRAM Access Ratio

DRAM access ratio 분석에서는 이러한 경향이 더욱 분명하게 드러났다. `split_transpose`는 execution time 비중 이상으로 높은 DRAM access 비중을 보였으며, 일부 설정에서는 전체 DRAM access의 30~40% 수준을 차지하였다. 이는 해당 kernel이 연산량 자체는 크지 않음에도 tensor layout 변환 과정에서 과도한 memory traffic을 유발하는 전형적인 memory-bound kernel임을 보여준다. 특히 sequence length가 길어질수록 이 비중이 빠르게 증가하였는데, 이는 split 및 transpose 연산이 input token 수 증가에 비례하여 memory movement를 확대시키기 때문이다. 따라서 이 kernel을 제거하거나 GEMM과

통합하는 방식은 단순한 kernel 개수 축소를 넘어, attention stage 전체의 메모리 병목을 구조적으로 완화하는 효과를 기대할 수 있다.

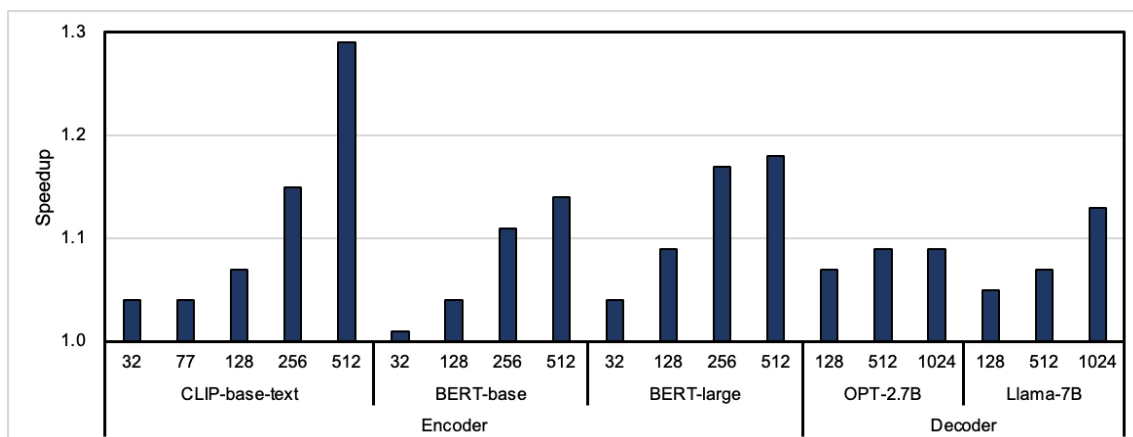


그림 6. 최종 Speedup

최종 speedup 결과 역시 앞선 분석과 일관된 경향을 나타낸다. QKV Fusion 적용 시 모든 모델에서 baseline 대비 1.0 이상의 성능 향상이 관찰되었으며, 특히 sequence length가 증가할수록 speedup이 커지는 경향이 뚜렷하게 확인되었다. CLIP-base-text의 경우 긴 sequence에서 최대 약 1.29배의 성능 향상이 관찰되었고, BERT-base 및 BERT-large에서도 1.1배 이상의 개선이 확인되었다. OPT-2.7B와 Llama-7B와 같은 decoder 계열에서는 개선 폭이 상대적으로 완만하였으나, 일관된 speedup이 유지되었다. 이는 split_transpose의 execution time 및 DRAM access 비중이 큰 조건일수록 fusion의 효과가 직접적으로 확대됨을 의미한다. 반대로 짧은 sequence에서는 split_transpose 자체의 절대 비용이 작기 때문에 speedup이 제한적으로 나타났는데, 이는 해당 구간에서는 fixed launch overhead와 GEMM 중심의 계산 비용이 전체 latency에서 차지하는 비율이 더 크기 때문이다. 또한 hidden dimension이 큰 decoder 계열에서는 GEMM의 계산 비중이 상대적으로 높으므로, split_transpose 제거에 따른 이득이 encoder 계열에 비해 다소 완만하게 나타난 것으로 해석할 수 있다.

한편, GEMM + Residual Add + LayerNorm + Bias Add Fusion은 현재 구현 및 최적화가 진행 중이며, 이에 대한 실험 설계 또한 요구사항에 포함된다. 이 feature는 QKV Fusion과 달리 단순 tensor transformation 제거에 그치지 않고, GEMM epilogue에 element-wise 연산과 reduction 기반 normalization을 함께 포함해야 하므로 구현 난도가 높다. 따라서 실험에서는 단순 latency뿐 아니라, fused kernel 내부에서의 reduction efficiency, shared memory 활용도, synchronization overhead, 그리고 numerical stability까지 함께 검토해야 한다. 예상되는 결과는 attention output projection 이후 또는 MLP projection 이후에 연속적으로 호출되던 residual add, bias add, layer normalization kernel이 통합됨에 따라 kernel launch 수와 intermediate tensor write/read가 감소하고, 특히 edge GPU 환경에서 더 큰 latency 개선이 나타나는 것이다. 다만 layer normalization의 reduction 비용 자체가 완전히 제거되는 것은 아니므로, QKV Fusion과 같은 직접적인 구조적 이득보다는 memory traffic 감소와 launch overhead 완화에 기반한 점진적인 성능 향상 형태로 나타날 가능성이 높다.

B. Overall System Architecture

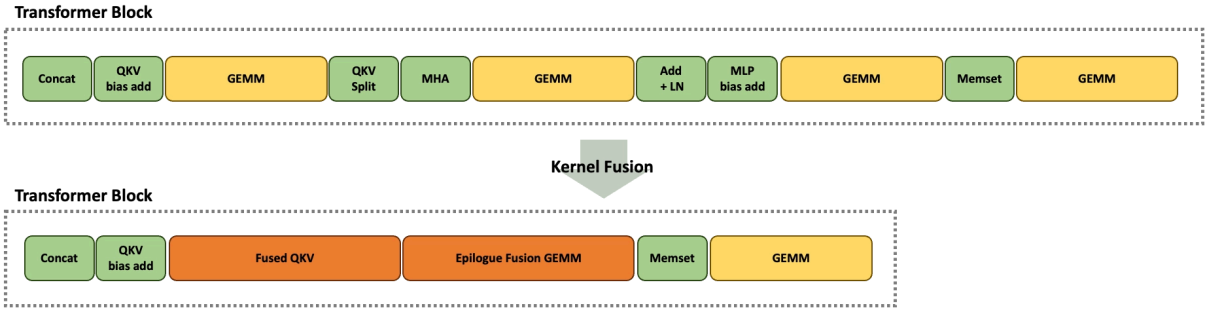


그림 6. 커널 퓨전을 적용한 이후의 전체 흐름

전체 시스템의 처리 흐름은 다음과 같다. 먼저, TensorRT 기반 멀티모달 모델을 baseline으로 설정하고, Nsight Systems를 이용하여 Transformer block 내부의 kernel 호출 순서와 실행 시간을 분석한다. 이후 Nsight Compute를 통해 각 kernel의 arithmetic intensity, DRAM access, memory throughput 특성을 분석하여 memory-bound kernel을 식별한다. 이러한 profiling 결과를 바탕으로, 본 연구에서는 Transformer block 내부에서 반복적으로 등장하는 비효율적인 연산 흐름을 대상으로 두 가지 핵심 fusion 기법, 즉 QKV Fusion과 GEMM Epilogue Fusion을 적용한다. 마지막으로, fused kernel이 적용된 시스템과 기존 TensorRT baseline을 비교하여 execution time, DRAM access, 그리고 latency 개선 효과를 정량적으로 평가한다.

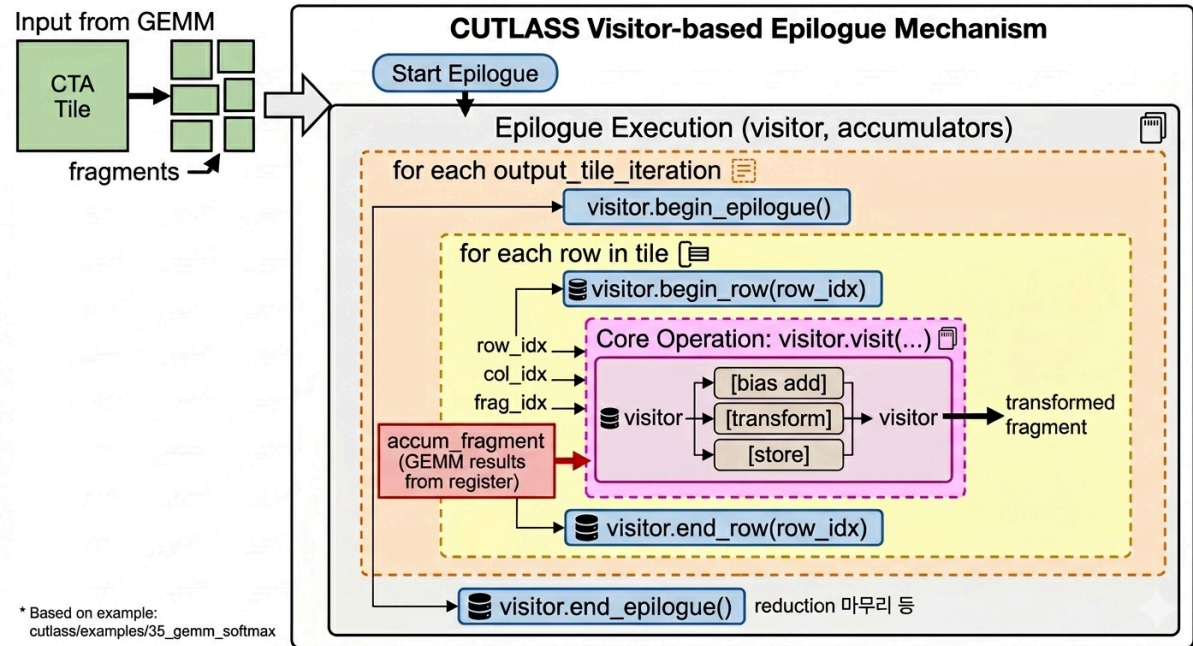


그림 7. CUTLASS 기반 Epilogue Fusion

시스템 아키텍처의 핵심은 CUTLASS 기반 GEMM kernel의 확장에 있다. 본 연구에서는 CUTLASS의 GEMM 구조를 기반으로 하되, GEMM 결과가 global memory로 기록되기 전에 epilogue 단계에서 추가 연산을 수행할 수 있도록 visitor 기반 epilogue processing 구조를 활용한다. CUTLASS의 epilogue는 accumulator에 저장된 GEMM 결과를 CTA tile 단위로 순회하면서 output fragment를 처리하는 구조를 가지며, 각 fragment에 대해 visitor가 순차적으로 호출되어 후처리를 수행한다. 구체적으로, epilogue 내부에서는 output tile iteration과 row loop를 따라 accumulator fragment가 visitor에 전달되며, visitor는 해당 fragment에 대해 bias addition, residual addition, tensor transform, reduction 기반 normalization, 그리고 최종 store 연산을 수행한다. 이와 같은 구조를 이용하면, 기존에는 별도의 CUDA kernel로 분리되어 있던 후처리 연산을 GEMM epilogue 내부에 통합할 수 있으므로, intermediate tensor를 global memory에 별도로 저장하지 않고도 연속적인 처리가 가능하다.

첨부한 kernel fusion 개념도는 이러한 시스템 재구성의 방향을 요약하여 보여준다. 기존 Transformer block은 `Concat`, `QKV bias add`, `GEMM`, `QKV split`, `MHA`, `GEMM`, `Add + LN`, `MLP bias add`, `GEMM`, `Memset`, `GEMM`과 같이 다수의 kernel이 순차적으로 호출되는 구조를 갖는다. 반면, 제안하는 시스템에서는 이 중 `QKV projection + split/transpose` 구간을 Fused QKV로 통합하고, `attention output projection + residual add + layer normalization + bias add` 구간을 Epilogue Fusion GEMM으로 통합한다. 그 결과, Transformer block 내부의 연산 흐름은 보다 긴 fused kernel 중심으로 재구성되며, 세분화된 kernel 호출 구조에서 발생하던 launch overhead와 memory traffic이 감소하게 된다. 즉, 본 시스템은 단순히 일부 연산을 묶는 수준이 아니라, Transformer block 내부의 데이터 이동 경로 자체를 축소하는 방향으로 전체 실행 구조를 재설계한 것이다.

제안하는 전체 시스템의 주요 설계 목표는 다음과 같이 정리할 수 있다. 첫째, memory-bound 특성이 강한 tensor transformation 및 post-processing kernel을 제거하거나 축소하여 DRAM access를 최소화한다. 둘째, GEMM 결과를 가능한 한 register 또는 on-chip memory 수준에서 후속 연산과 직접 연결함으로써 arithmetic intensity를 높인다. 셋째, 여러 개의 짧은 kernel을 소수의 fused kernel로 대체하여 kernel launch overhead를 줄인다. 넷째, 이러한 최적화가 특정 단일 모델에 국한되지 않도록, Transformer block 수준의 공통 연산 패턴을 기준으로 설계하여 CLIP, BERT, OPT, Llama 계열과 같은 다양한 encoder 및 decoder 구조에 적용 가능하도록 한다.

또한 본 시스템은 성능 개선뿐 아니라 실제 적용 가능성도 함께 고려한다. QKV Fusion은 attention preparation 단계에서 반복적으로 호출되는 split/transpose kernel을 제거함으로써 비교적 직접적인

latency 감소 효과를 기대할 수 있으며, sequence length가 증가할수록 그 효과가 더욱 크게 나타날 것으로 예상된다. GEMM Epilogue Fusion은 residual add, bias add, layer normalization과 같은 후처리 연산을 global memory round-trip 없이 수행하게 하므로, 특히 kernel launch overhead와 memory access 비용이 상대적으로 크게 작용하는 edge GPU 환경에서 유의미한 이점을 제공할 것으로 기대된다. 따라서 본 연구는 두 fusion 기법을 개별적으로 적용하는 데 그치지 않고, Transformer block 내부에서 연속적으로 결합하여 전체 block latency를 줄이는 방향으로 시스템을 구성한다.

종합하면, 본 연구의 전체 시스템은 profiling 기반 병목 식별, CUTLASS visitor 기반 fused epilogue 설계, QKV 및 post-processing kernel fusion, 그리고 baseline 대비 성능 검증의 네 단계로 구성된다. 이를 통해 기존 Transformer block의 fragmented execution structure를 fused execution structure로 변환하고, global memory access와 kernel launch overhead를 동시에 줄임으로써 멀티모달 모델 추론의 지연시간을 효과적으로 단축하는 것을 목표로 한다. 최종 산출물로는 멀티모달 모델의 Transformer block에 최적화된 kernel fusion 코드와, 이에 대한 설계·구현·실험 결과를 정리한 대한전자공학회 투고 논문을 제시한다.

C. Design of Major Engines and Features

1. QKV Split/Transpose Fusion

본 모듈은 Transformer attention의 QKV projection 단계에서 기존에 독립적으로 수행되던 GEMM 연산과 QKV split+transpose 연산을 단일 CUDA 커널에 통합하여 수행하는 Fusion 구조를 제안한다. 이를 통해 GEMM의 출력 activation tensor가 DRAM에 write되었다가 다시 read되는 불필요한 왕복을 제거하고, kernel launch overhead 및 메모리 접근 비용을 감소시키는 것을 목표로 한다.

기존 구조에서는 QKV projection을 수행하는 GEMM kernel이 $[M, 3 \cdot H \cdot D]$ 형태의 출력 tensor를 global memory에 write한 뒤, 별도의 split+transpose kernel이 이를 다시 read하여 $Q[H, M, D]$, $K[H, M, D]$, $V[H, M, D]$ 세 개의 tensor로 재배치하는 과정을 거친다. 이 과정에서 동일한 activation tensor에 대해 한 차례의 write와 한 차례의 read가 추가적으로 발생하며, 두 개의 독립적인 kernel launch가 필요하다. 본 설계는 이러한 비효율성을 제거하기 위해 split+transpose 연산을 GEMM의 epilogue 단계에 fusion한 1-stage pipeline 형태로 연산을 재구성하였다.

상세 구현: Single-Kernel Epilogue Fusion Pipeline

전체 연산 과정은 CUTLASS 라이브러리의 EpilogueVisitor 구조를 확장하여 단일 단계로 수행한다. LayerNorm fusion이 global reduction의 필요성으로 인해 3-stage pipeline을 요구하는 것과 달리, QKV split+transpose는 각 element가 독립적으로 처리 가능한 element-wise reshape 연산이므로 별도의 synchronization kernel 없이 epilogue 단계에서 완전히 처리 가능하다.

Stage 1: GEMM Epilogue (Fused Split & Transpose)

QKV projection GEMM 수행 이후 각 Thread Block은 Epilogue 단계에서 다음 연산을 수행한다.

- 레지스터 내부에서 elementwise scaling ($\alpha \cdot \text{accum} + \beta \cdot C$) 수행 및 `fragment_D_` 버퍼에 저장
- Output tile coordinate로부터 (`qkv_idx`, `head_idx`, `dim_idx`) 변환
- 변환된 좌표를 통해 Q/K/V 중 해당하는 destination buffer 선택
- 128-bit vectorized store를 통해 transposed layout으로 직접 write

이 단계는 각 thread block이 독립적으로 수행하며, 별도의 cross-block synchronization 없이 단일 kernel launch 내에서 전체 연산이 완료된다.

Performance Analysis

1. DRAM Memory Traffic Reduction

DRAM access는 커널 경계(kernel boundary)에서 발생하는 activation tensor의 read/write 연산을 기준으로 계산하였다.

기존 구조에서는 GEMM과 split+transpose kernel 사이에서 intermediate activation tensor $[M, 3 \cdot H \cdot D]$ 가 다음과 같은 memory access를 거치게 된다.

- GEMM output write ($[M, 3 \cdot H \cdot D]$)
- Split+transpose input read ($[M, 3 \cdot H \cdot D]$)
- Split+transpose output write (Q, K, V 각각 $[H, M, D]$)

이에 따라 intermediate activation tensor는 총 2회의 DRAM-level read/write access를 갖는다(Q, K, V 최종 write는 두 구조 모두에서 공통이므로 분석에서 제외).

제안된 구조에서는 split+transpose가 독립적인 kernel로 분리되지 않고 GEMM epilogue 단계에 fusion되어 수행된다. 이로 인해 intermediate tensor는 다음과 같이 제한된다.

- GEMM accumulator register fragment → Q/K/V global memory write (단일 경로)

즉 intermediate activation tensor $[M, 3 \cdot H \cdot D]$ 는 global memory에 한 번도 materialize되지 않으며, GEMM accumulator의 register fragment 상태에서 직접 transposed layout으로 store된다. 따라서 intermediate-level DRAM access는 0회로 감소한다.

※ 본 분석은 intermediate tensor 기준이며, 입력 A, B와 최종 출력 Q, K, V에 대한 DRAM access는 두 구조에서 동일하다.

2. Arithmetic Intensity 개선

Split+transpose 연산은 본질적으로 memory-bound reshape 연산이며, FLOP 대비 memory traffic 비율이 매우 낮다. 본 설계에서는 이러한 memory-bound 연산이 compute-bound GEMM의 execution path 내부로 통합됨에 따라, intermediate tensor가 on-chip memory hierarchy(register) 수준에서만 처리된다.

이로 인해 동일 연산량 대비 global memory access 비율이 감소하며, compute utilization 관점에서 arithmetic intensity가 증가하는 효과를 가진다. 또한 별도의 reshape kernel이 차지하던 memory bandwidth를 다른 단계의 연산에 할당할 수 있게 된다.

3. Kernel Launch Overhead Reduction

기존 구조에서는 GEMM, split+transpose로 이어지는 2개의 independent kernel launch가 필요하다. 제안 구조에서는 다음과 같이 execution pipeline이 재구성된다.

- GEMM + epilogue fusion (단일 kernel)

이를 통해 kernel launch 수가 2회에서 1회로 감소하며, 특히 batch size가 작거나 sequence length가 짧은 환경에서 두드러지는 launch overhead가 완화된다.

4. Design Trade-off

QKV split-transpose는 element-wise 연산 특성을 가지므로 LayerNorm과 달리 global reduction이 불필요하다. 그러나 GEMM의 출력 tile은 row-major $[M, N]$ 좌표계로 생성되는 반면, 목표 layout은 transposed된 $[H, M, D]$ 이므로 다음과 같은 trade-off가 존재한다.

- GEMM의 자연스러운 row-major output 접근 패턴이 깨지며, $Q/K/V$ 에 대한 store가 strided pattern으로 변환된다
- N 차원이 $[3, H, D]$ 로 reinterpret되며, thread당 좌표 변환 연산이 추가된다

본 설계에서는 다음과 같은 최적화를 통해 이러한 overhead를 최소화하였다.

- thread block 내 인접 thread들이 연속된 N 좌표를 처리하는 ThreadMap 특성을 활용하여, 인접 thread가 동일 head 내 연속된 dim_idx 를 처리하도록 매핑 → coalesced store 보장
- $head_dim$ 및 $H \cdot D$ 가 2의 거듭제곱인 경우 integer division을 bit shift로 대체하여 좌표 변환 비용 제거
- visit() 단계에서는 결과를 register fragment에만 buffering하고, end_step() 단계에서 128-bit vectorized store(`arch::global_store`)를 수행하여 store throughput 극대화

이 구조는 memory traffic reduction과 store coalescing을 동시에 고려한 설계이다.

5. Summary

본 설계는 QKV projection 단계의 split-transpose 연산을 GEMM execution path에 통합하여 수행함으로써, intermediate activation tensor $[M, 3 \cdot H \cdot D]$ 의 DRAM materialization을 완전히 제거하고 kernel launch overhead를 절반으로 감소시킨다.

2. GEMM Epilogue Fusion

본 모듈은 기존에 독립적으로 수행되던 Bias Add, Residual Add, Layer Normalization 연산을 GEMM 커널의 Epilogue 및 Mainloop 단계에 통합하여 수행하는 Fusion 구조를 제안한다. 이를 통해 중간 활성화

텐서의 불필요한 DRAM 왕복을 줄이고, 전체 추론 파이프라인의 메모리 접근 비용을 감소시키는 것을 목표로 한다.

기존 구조에서는 GEMM 연산과 정규화 연산이 서로 독립된 커널로 분리되어 실행되며, 이로 인해 다수의 메모리 read/write 및 커널 launch overhead가 발생한다. 본 설계는 이러한 비효율성을 줄이기 위해 3-stage pipeline 형태로 연산을 재구성하였다.

상세 구현: 3-Stage Pipeline

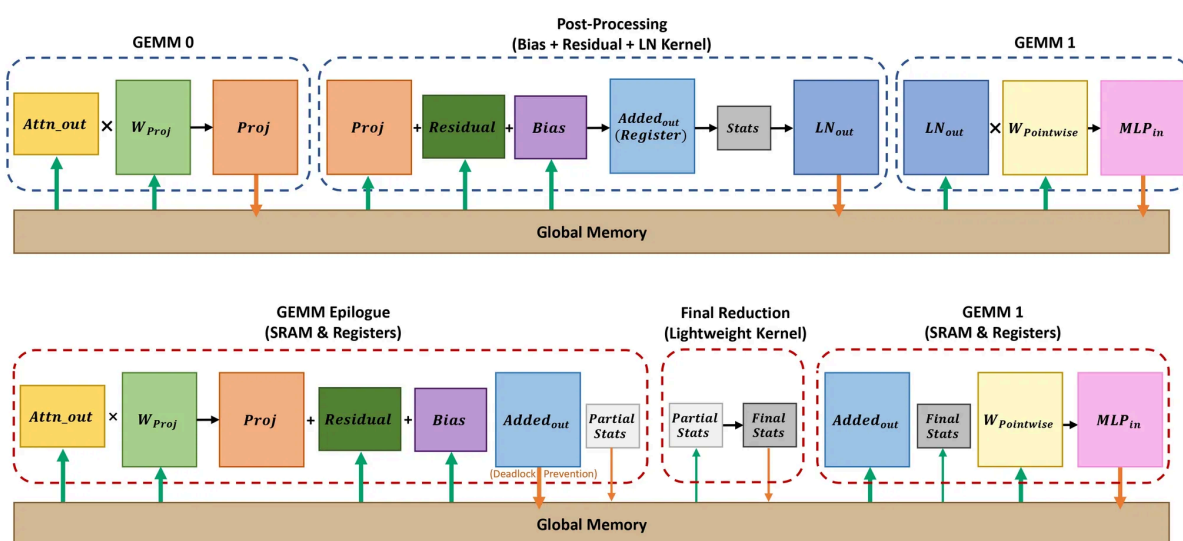


그림 8. 상단: Unfused, 하단: Fused

전체 연산 과정은 CUTLASS 라이브러리의 구조를 최적화하여 다음과 같은 3단계로 분해하여 수행한다.

Stage 1: GEMM 0 Epilogue (Partial Reduction)

Attention Output Projection(GEMM 0) 수행 이후 각 Thread Block은 Epilogue 단계에서 다음 연산을 수행한다.

- 레지스터 내부에서 Bias Add 및 Residual Add 수행
- 블록 단위 partial sum 및 squared sum 계산
- Partial statistics (sum, squared sum)을 Global Memory의 임시 buffer에 저장
- bias/residual이 반영된 activation tensor(Added_out)를 global memory에 write

이 단계는 각 thread block이 독립적으로 수행하며, 이후 global reduction을 위한 중간 통계값을 생성하는 역할을 한다.

Stage 2: Global Reduction Kernel (ApplyFinalReduction)

Stage 1에서 저장된 partial statistics를 기반으로 전체 activation tensor에 대한 global statistics(mean, variance)를 계산한다.

- 모든 thread block의 partial sum 및 squared sum을 reduction
- 전체 tensor에 대한 mean 및 variance 계산
- 결과 statistics를 global memory에 저장

본 단계는 GEMM 0 이후 수행되는 별도의 synchronization kernel로, LayerNorm 수행에 필요한 global statistics을 생성한다.

Stage 3: GEMM 1 Mainloop (Normalization Fusion)

MLP의 첫 번째 fully-connected layer(GEMM 1) 수행 시, Layer Normalization은 독립적인 kernel으로 수행되지 않으며, GEMM 1의 epilogue 또는 mainloop 진입 단계에 fusion되어 수행된다.

- GEMM 1이 input activation(Added_out)과 global statistics(mean, variance)를 함께 load
- load된 데이터를 기반으로 normalization 연산 수행
- 정규화된 activation을 GEMM 1 연산에 직접 사용
- 별도의 normalized tensor write-back 없이 computation pipeline 내에서 처리

본 구조는 standalone LayerNorm kernel을 제거하고, 이를 GEMM 1 실행 경로에 fusion하여 처리한다.

Performance Analysis

1. DRAM Memory Traffic Reduction

DRAM access는 커널 경계(kernel boundary)에서 발생하는 activation tensor의 read/write 연산을 기준으로 계산하였다.

기존 구조에서는 GEMM 0, LayerNorm, GEMM 1 사이에서 intermediate activation tensor가 여러 단계의 memory read/write를 거치게 된다. 구체적으로 activation tensor 기준으로 다음과 같은 memory access가 발생한다.

- GEMM 0 output write
- LayerNorm input read
- LayerNorm output write
- GEMM 1 input read

이에 따라 activation tensor는 총 4회의 DRAM-level read/write access를 갖는다.

제안된 구조에서는 LayerNorm이 독립적인 kernel로 분리되지 않고 GEMM epilogue 및 mainloop 단계에 fusion되어 수행된다. 이로 인해 activation tensor는 다음과 같은 경로로 제한된다.

- GEMM 0 epilogue write (Added_out)
- GEMM 1 mainloop read (Added_out)

따라서 activation-level DRAM access는 2회로 감소한다.

※ 본 분석은 activation tensor 기준이며, LayerNorm 수행을 위한 mean/variance statistics는 별도의 reduction kernel에서 생성 및 DRAM 접근이 발생한다.

2. Arithmetic Intensity 개선

LayerNorm과 같은 memory-bound 연산이 GEMM execution path 내부로 통합됨에 따라, intermediate tensor가 on-chip memory hierarchy (register and shared memory) 수준에서 처리되는 구간이 증가한다.

이로 인해 동일 연산량 대비 global memory access 비율이 감소하며, compute utilization 관점에서 arithmetic intensity가 증가하는 효과를 가진다.

3. Kernel Launch Overhead Reduction

기존 구조에서는 GEMM, Bias/Residual Add, LayerNorm, GEMM으로 이어지는 다수의 independent kernel launch가 필요하다.

제안 구조에서는 다음과 같이 execution pipeline이 재구성된다.

- GEMM 0 + epilogue fusion
- lightweight reduction kernel
- GEMM 1 + fused normalization

이를 통해 kernel launch 수가 감소하며, CPU-GPU synchronization overhead가 완화된다.

4. Design Trade-off

LayerNorm은 global reduction이 필요한 연산 특성을 가지므로, 완전한 register-resident execution은 불가능하다. 이에 따라 본 설계에서는 다음과 같은 trade-off를 채택하였다.

- block-level statistics computation in GEMM 0 epilogue
- global statistics computation via separate reduction kernel
- normalization fused into GEMM 1 execution path

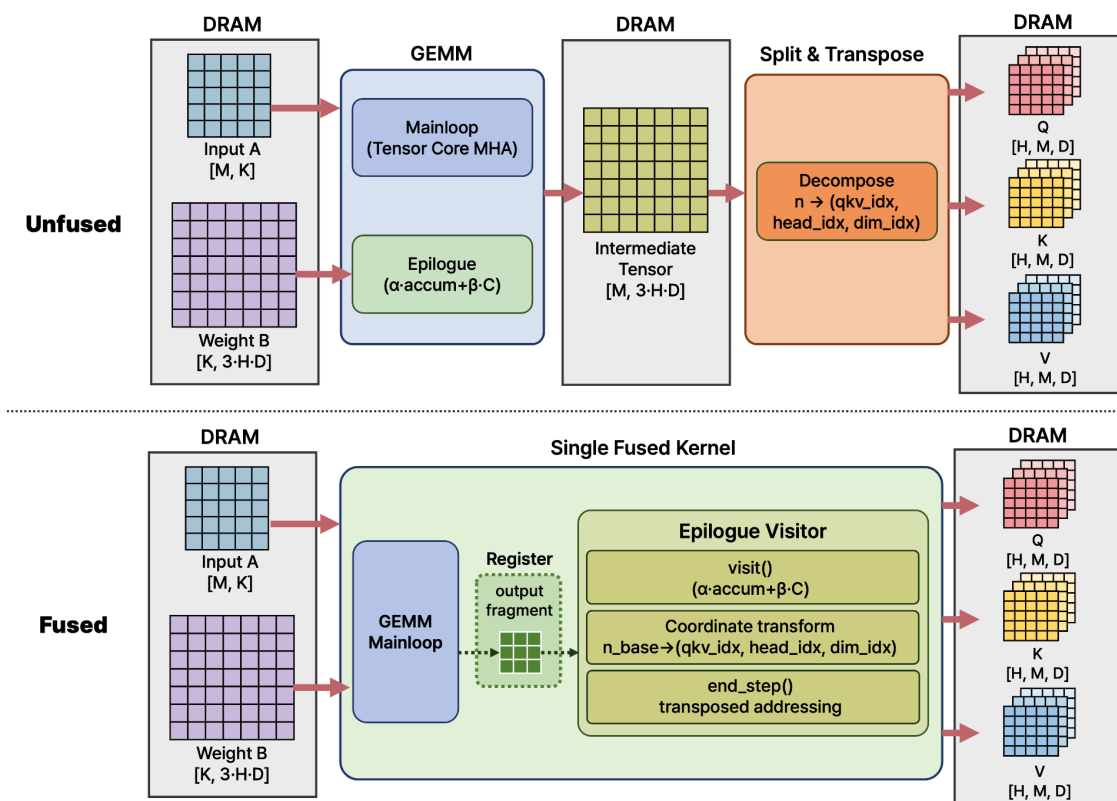
이 구조는 memory traffic reduction과 synchronization correctness를 동시에 고려한 설계이다.

5. Summary

본 설계는 Bias Add, Residual Add, LayerNorm을 GEMM execution path에 통합하여 수행함으로써, intermediate activation tensor의 DRAM access 및 kernel launch overhead를 감소시킨다.

D. Implementation of Major Features

1. QKV Split/Transpose Fusion



코드 구조 및 구현 흐름

본 절에서는 제안한 GEMM QKV Split/Transpose Fusion 구조가 실제 CUDA/CUTLASS 코드 상에서 어떻게 구현되는지를 설명한다. 구현은 CUTLASS 2.x의 epilogue visitor pattern을 활용하였으며, 크게 다음과 같은 구성 요소로 이루어진다.

- `gemm_with_qkv_split.h`
- `epilogue_visitor_qkv_split.h`
- `gemm_qkv_split.cu`
- `benchmark_qkv.cu`

각 파일은 single-kernel fusion pipeline의 특정 역할을 담당하며, kernel orchestration, epilogue visitor 구현, 검증 및 성능 측정 구조를 단계적으로 수행한다.

1) `epilogue_visitor_qkv_split.h`

`epilogue_visitor_qkv_split.h`는 본 fusion 구조의 핵심 구현 파일로, GEMM epilogue 단계에서 수행되는 split-transpose 로직을 정의하는 `EpilogueVisitorQKVSplit` 클래스를 포함한다. 해당 클래스는 기존 CUTLASS epilogue visitor 인터페이스를 확장하여 register 수준에서 transposed layout으로의 직접 store를 수행하도록 구현하였다.

구현의 주요 특징은 다음과 같다.

- Buffer-then-Store Pattern

기존의 단순 epilogue visitor 구현은 `visit()` 단계에서 elementwise 연산과 store를 동시에 수행하지만, 본 구현에서는 두 단계를 분리하였다.

```
// visit(): elementwise 연산 결과를 register fragment에 buffering
void visit(int iter_idx, int row_idx, int column_idx, int frag_idx,
           AccumulatorFragment const &accum) {
    OutputVector &output = reinterpret_cast<OutputVector*>(&fragment_D_)[frag_idx];
    output = elementwise_(accum, source_vector);
    // No store here - buffered for end_step()
}
```

`visit()` 단계에서는 GEMM accumulator에 대한 elementwise scaling($\alpha \cdot \text{accum} + \beta \cdot C$) 결과를 `fragment_D_` register fragment에 저장만 수행하고, 실제 global memory store는 `end_step()` 단계에서 일괄적으로 수행한다.

이 분리 구조를 통해:

- per-element store 대신 vectorized store 가능

- 좌표 변환 연산을 vector 단위로 amortize
- register pressure와 instruction-level parallelism 균형 최적화

효과를 얻을 수 있다.

- 128-bit Vectorized Store

end_step() 단계에서는 buffer된 결과를 CUTLASS의 `arch::global_store` intrinsic을 사용하여 128-bit 단위로 store한다.

```
arch::global_store<OutputVector, sizeof(OutputVector)>(
    data,
    (void*)(ptr_dst + out_offset + batch_offset_),
    true
);
```

`OutputVector`는 `kElementsPerAccess` (half 타입 기준 8개)의 element로 구성된 vector 타입이며, 이를 통해 thread당 한 번의 store instruction으로 8개 element를 한꺼번에 write한다. 인접 thread들이 인접한 메모리 영역을 access하도록 ThreadMap을 활용함으로써 coalesced store가 보장된다.

- Bit-shift 기반 좌표 변환

N 차원 좌표를 (`qkv_idx`, `head_idx`, `dim_idx`)로 분해하기 위해서는 `head_dim` 및 $H \cdot D$ 에 대한 integer division 및 modulo 연산이 필요하다. 그러나 transformer 모델에서는 이들이 대부분 2의 거듭제곱이므로, 이를 활용하여 bit shift 및 bit mask로 대체하였다.

```
if (params_.use_bitshift) {
    qkv_idx = n_base >> params_.hd_log2;
    n_local = n_base & (params_.hd - 1);
    head_idx = n_local >> params_.head_dim_log2;
    dim_idx = n_local & (head_dim - 1);
} else {
    qkv_idx = n_base / params_.hd;
    n_local = n_base - qkv_idx * params_.hd;
    head_idx = n_local / head_dim;
    dim_idx = n_local - head_idx * head_dim;
}
```

Params 생성 시점에 $H \cdot D$ 및 `head_dim`의 power-of-2 여부를 검사하여 `use_bitshift` flag를 설정하며, runtime에는 single branch로 두 경로를 선택한다. 또한 vector store 단위로 좌표 변환을 1회만 수행하므로, vector 내부 8개 element는 동일한 변환 비용을 amortize한다.

- Transposed Store Addressing

변환된 좌표 (`qkv_idx`, `head_idx`, `dim_idx`)와 row coordinate `m`을 기반으로 `[H, M, D]` row-major layout에서의 offset을 계산한다.

```
int out_offset = head_idx * stride_head + m * head_dim + dim_idx;

ElementOutput *ptr_dst;
switch (qkv_idx) {
    case 0: ptr_dst = params_.ptr_Q; break;
    case 1: ptr_dst = params_.ptr_K; break;
    default: ptr_dst = params_.ptr_V; break;
}
```

`qkv_idx`에 따라 `Q/K/V` 세 개의 destination pointer 중 하나를 선택하며, branch divergence는 warp 내에서 인접 thread들이 동일한 `qkv_idx`를 처리하도록 ThreadMap이 보장되므로 최소화된다.

- Bounds Check

Vector store 수행 전에 row 및 column 경계를 확인하여 out-of-bounds access를 방지한다.

```
bool guard = (m < extent_.row()) &&
              (n_base + kElementsPerAccess - 1 < extent_.column());
```

Vector 단위로 guard를 검사함으로써 per-element guard 대비 분기 비용을 1/8로 감소시켰다.

2) `gemm_with_qkv_split.h`

`gemm_with_qkv_split.h`는 single-kernel fusion pipeline의 orchestration을 담당하는 `GemmQKVSplit` 클래스를 정의한다.

LayerNorm fusion의 `GemmLayernorm` 클래스가 3-stage pipeline orchestration을 수행하는 것과 달리, 본 구조는 별도의 reduction kernel이 필요 없으므로 단일 kernel launch로 전체 연산이 완료되는 간소한 구조를 갖는다.

해당 클래스는 다음 구성 요소를 template parameter로 받아 fusion kernel을 합성한다.

- `DefaultGemm` kernel (mainloop `Mma` 및 base `Epilogue` 제공)
- `EpilogueVisitorQKVSplit` (custom epilogue visitor)
- `GemmWithEpilogueVisitor` (kernel dispatch wrapper, example 35로부터 재사용)

```

using DefaultGemmKernel = typename cutlass::gemm::kernel::DefaultGemm<...>::GemmKernel;

using EpilogueVisitor = EpilogueVisitorQKVSplit
    ThreadblockShape,
    DefaultGemmKernel::kThreadCount,
    typename DefaultGemmKernel::Epilogue::OutputTileIterator,
    ElementCompute, ElementCompute,
    EpilogueFunctorOp
>;

using Epilogue = typename EpilogueWithVisitorFromExistingEpilogue
    EpilogueVisitor,
    typename DefaultGemmKernel::Epilogue
>::Epilogue;

using GemmKernel = gemm::kernel::GemmWithEpilogueVisitor
    typename DefaultGemmKernel::Mma,
    Epilogue,
    ThreadblockSwizzle
>;

```

`GemmQKVSplit::run()` 메서드는 grid/block dimension 계산, dynamic shared memory 설정, kernel launch를 단일 흐름으로 수행한다.

```

cutlass::Kernel<GemmKernel><<<gemm_grid, gemm_block, gemm_smem_size, stream>>>(
    params_.gemm);

```

해당 launch는 GEMM mainloop, elementwise scaling, QKV split·transpose를 모두 포함하는 단일 fused kernel을 실행한다. 또한 **Arguments** 구조체는 GEMM 입력 tensor(**A, B, C**), dummy output tensor(**D**, 실제로는 write되지 않음), QKV 출력 tensor 포인터(**Q, K, V**), attention 구성(**num_heads, head_dim**), 그리고 각 tensor의 batch stride를 통합하여 관리하며, 내부적으로 GEMM kernel arguments와 EpilogueVisitor arguments를 합성한다. Batched execution을 지원하기 위해 **GemmBatchedIdentityThreadblockSwizzle**을 사용하였다.

3) `gemm_qkv_split.cu`

`gemm_qkv_split.cu`는 fused kernel의 동작 및 numerical correctness를 검증하기 위한 테스트 코드이다.

(1) CPU Reference Implementation

검증 코드에는 fused kernel의 출력을 비교하기 위한 CPU reference 구현이 포함되어 있다.

- 단순 triple-nested loop로 GEMM 연산을 수행하여 `D_ref[M, N]`을 생성

- 동일한 좌표 변환 $\text{logic}(n / \text{hd}, n \% \text{hd}, n_local / D, n_local \% D)$ 을 적용하여 Q_ref, K_ref, V_ref 생성
- $\alpha \cdot \text{accum} + \beta \cdot C$ scaling을 명시적으로 적용

이 reference는 fused kernel의 좌표 변환 로직이 의도된 mathematical mapping과 일치함을 확인하는 ground truth 역할을 한다.

(2) Per-batch Element-wise Verification

Q, K, V 각각에 대해 모든 element를 reference와 비교하며, tolerance(기본값 $1e-2$)를 초과하는 mismatch를 카운트한다.

```
if (std::abs(float(gpu_Q[i]) - Q_ref[i]) > options.tolerance) {
    if (errors_q < 5) { /* 처음 5개 mismatch 출력 */ }
    errors_q++;
}
```

이를 통해 vectorized store 및 bit-shift 좌표 변환이 모든 element에 대해 정확하게 동작함을 검증한다.

(3) Performance Profiling

검증 통과 후 iterations 횟수만큼 fused kernel을 반복 실행하여 TFLOPS 및 effective bandwidth를 측정한다.

4) benchmark_qkv.cu

benchmark_qkv.cu는 baseline 구조와 제안한 fused pipeline의 성능을 공정하게 비교하기 위한 benchmark 코드이다. 단순 baseline 대비 measurement에 더해, fair comparison을 위해 세 가지 구성을 비교한다.

(1) V1 (Original Unfused Baseline)

GemmBatched device API와 별도의 split+transpose CUDA kernel을 순차 실행하는 구성이다.

```
GemmBatched gemm_op;
// ...
gemm_op();
split_transpose_kernel<<<grid, threads>>>(...);
```

이 구성에서 intermediate tensor block_D는 GEMM에 의해 write되고 split kernel에 의해 read된다.

(2) V1.5 (Fair Unfused Baseline)

`GemmBatched` 대신 V2와 동일한 GEMM template (`Gemm`)을 사용하고 batch마다 별도로 launch하는 구성이다.

이 baseline은 fusion의 효과를 GEMM template 선택의 효과로부터 분리하기 위한 것이다. V1의 `GemmBatched`는 작은 batch에서 비효율적이라는 점이 nsys profiling을 통해 확인되었으며, V1과 V2의 단순 비교는 fusion 효과와 GEMM template 효과를 혼동할 수 있다. V1.5는 V2와 GEMM template을 일치시킴으로써, 두 구성의 차이가 오로지 fusion 자체로부터 기인하도록 설계되었다.

(3) V2 (Proposed Fused Pipeline)

`GemmQKVSplit`을 사용하여 GEMM과 split-transpose를 단일 kernel로 실행하는 제안 구성이다.

(4) Numerical Verification

Benchmark 코드 시작 시 세 구성의 출력을 모두 비교하여 numerical correctness를 검증한다.

```
if (std::abs(float(h_Q1[i]) - float(h_Q2[i])) > tol || ...) errs_v1++;  
if (std::abs(float(h_Q15[i]) - float(h_Q2[i])) > tol || ...) errs_v15++;
```

V1, V1.5, V2의 출력이 모두 동일한 결과를 생성하는지 확인한 이후에만 timing 측정을 수행한다.

(5) Speedup Decomposition

측정 결과를 다음과 같이 분해하여 출력한다.

- GEMM template effect: V1 → V1.5 (fusion과 무관한 template 선택 효과)
- Pure fusion effect: V1.5 → V2 (fusion 자체의 효과)
- Combined effect: V1 → V2 (두 효과의 합)

이러한 분해를 통해 보고된 speedup이 실제 fusion 기여분과 부수적 요인을 명확히 구분하여 제시할 수 있다.

2. GEMM Epilogue Fusion

코드 구조 및 구현 흐름

본 절에서는 제안한 GEMM Epilogue Fusion 구조가 실제 CUDA/CUTLASS 코드 상에서 어떻게 구현되는지를 설명한다. 구현은 크게 다음과 같은 구성 요소로 이루어진다.

- `gemm_with_layernorm.h`
- `gemm_with_epilogue_visitor.h`
- `benchmark_gemm_layernorm.cu`

각 파일은 3-stage pipeline의 특정 역할을 담당하며, GEMM Epilogue Fusion, Global Reduction, GEMM Mainloop Fusion 구조를 단계적으로 수행한다.

`gemm_with_layernorm.h`

`gemm_with_layernorm.h`는 전체 3-stage LayerNorm fusion pipeline을 구성하는 핵심 구현 파일이다. 해당 파일에는 partial statistics reduction, epilogue fusion visitor, pipeline orchestration을 위한 주요 클래스들이 포함되어 있다.

(1) `ApplyFinalReduction`

`ApplyFinalReduction`은 Stage 2(Global Reduction Kernel)를 수행하는 lightweight CUDA reduction kernel이다. 해당 kernel은 Stage 1에서 계산된 partial statistics를 reduction하여 최종 mean 및 variance를 계산한다.

구현 과정은 다음과 같다.

- 각 thread block이 저장한 partial sum 및 squared sum을 global memory로부터 load
- reduction 연산을 통해 전체 activation tensor에 대한 mean 계산
- $Var = E[X^2] - (E[X])^2$ 공식을 이용하여 variance 계산
- 계산된 mean 및 variance를 global statistics buffer에 저장

추가적으로, 부동소수점 정밀도 오차로 인해 variance가 음수 값으로 계산되는 상황을 방지하기 위한 numerical stabilization logic이 포함되어 있다.

```
if (var < 0) {
    var = 0;
}
```

해당 예외 처리는 floating-point truncation error에 의해 `square_mean < mean * mean` 상황이 발생할 경우를 대비한 방어 로직이며, 이후 수행되는 square-root 연산에서 NaN 발생을 방지한다.

또한 구현에는 shifted variance 기반 LayerNorm과 standard LayerNorm 모두를 지원하기 위한 분기 구조가 포함되어 있어, 다양한 LayerNorm variant에 대응 가능하도록 설계하였다.

(2) EpilogueVisitorLayerNorm

`EpilogueVisitorLayerNorm`은 Stage 1(GEMM 0 Epilogue Fusion)을 수행하는 custom epilogue visitor 클래스이다.

기존 CUTLASS epilogue visitor 구조를 확장하여 Bias Add, Residual Add, partial statistics computation을 register 수준에서 수행하도록 구현하였다.

구현의 주요 특징은 다음과 같다.

- Register-level Bias & Residual Fusion

GEMM 결과는 epilogue 단계에서 accumulator register fragment 형태로 유지된다.

`EpilogueVisitorLayerNorm`은 해당 accumulator를 직접 참조하여 Bias Add 및 Residual Add를 수행한다.

이를 위해 다음과 같은 구성 요소가 추가되었다.

- Bias/Residual tensor iterator
- register fragment buffer
- vectorized register accumulation loop

이 구조를 통해 intermediate activation tensor를 global memory에 write하기 전에 register 수준에서 후처리를 수행할 수 있다.

또한 Bias tensor는 stride를 0으로 설정하여 broadcasting 형태로 접근하도록 구현하였으며, 이를 통해 불필요한 memory access를 최소화하였다.

- Partial Statistics Computation

Bias 및 Residual이 반영된 activation tensor에 대해 partial sum 및 squared sum을 계산한다.
이 과정에서 warp-level reduction을 이용하여 thread 간 partial statistics aggregation을 수행한다.
구현에는 CUDA warp shuffle intrinsic인 `__shfl_xor_sync`가 사용되며, shared memory를 경유하지 않고 register-to-register communication 기반 reduction을 수행한다.

이를 통해:

- shared memory traffic 감소
- synchronization overhead 감소
- partial reduction latency 감소

효과를 얻을 수 있다.

- Tile Indexing Logic

다중 tile 환경에서 partial statistics 저장 주소 충돌을 방지하기 위해 tile indexing logic을 수정하였다.
기존 단순 block index 기반 방식 대신:

```
n_tile_idx = thread_offset_.column() / ThreadblockShape::kN;
```

형태의 column-coordinate 기반 tile indexing을 적용하여, 각 thread block이 global memory의 올바른 statistics buffer 영역에 접근하도록 구현하였다.

이를 통해 multi-tile execution 환경에서도 partial statistics write collision이 발생하지 않도록 하였다.

(3) GemmLayernorm 클래스

`GemmLayernorm` 클래스는 전체 3-stage fusion pipeline의 orchestration 역할을 수행한다.

해당 클래스는:

1. GEMM 0 Epilogue Fusion
2. Final Reduction

3. GEMM 1 Mainloop Fusion

단계를 순차적으로 구성 및 실행하며, 각 stage 간 intermediate tensor 및 statistics buffer를 관리한다.

- Stage 1: GEMM 0 Epilogue Fusion

첫 번째 단계에서는 `GemmEpilogueFusion` kernel이 실행된다.

해당 kernel은:

- GEMM 수행
- Bias Add
- Residual Add
- partial statistics computation

을 단일 execution flow 내에서 수행한다.

GEMM 결과는 global memory에 즉시 write되지 않고 accumulator register fragment에 유지되며, 이후 `EpilogueVisitorLayerNorm`이 해당 accumulator를 직접 참조하여 후처리를 수행한다

- Stage 2: Final Reduction

두 번째 단계에서는 `ApplyFinalReductionKernel`이 실행된다.

이 kernel은 Stage 1에서 생성된 partial statistics를 reduction하여 최종 mean 및 variance를 계산한다.

계산 결과는 이후 GEMM 1 단계에서 참조 가능한 global statistics buffer에 저장된다.

- Stage 3: GEMM 1 Mainloop Fusion

세 번째 단계에서는 `gemm_fusion_op` 기반 GEMM kernel이 실행된다.

해당 kernel은 GEMM mainloop 수행 과정에서:

- input activation tensor
- mean
- variance

를 함께 load하며, activation tensor가 register에 load된 직후 normalization을 수행한다.

정규화된 activation은 별도의 write-back 과정 없이 GEMM operand로 직접 사용된다.

이를 통해 standalone LayerNorm kernel을 제거하고, normalization을 GEMM execution path 내부에 통합하였다.

gemm_with_epilogue_visitor.h

`gemm_with_epilogue_visitor.h`는 Stage 1(GEMM 0 Epilogue Fusion)의 실제 CUDA execution kernel 구현을 포함한다.

핵심 클래스인 `GemmWithEpilogueVisitor`는:

- GEMM mainloop (`Mma`)
- custom epilogue (`Epilogue`)
- threadblock scheduling (`ThreadblockSwizzle`)

구성 요소를 결합하여 단일 CUDA kernel pipeline을 형성한다.

(1) GEMM Mainloop (`Mma`)

`mma(...)` 함수는 Tensor Core 기반 matrix multiplication을 수행한다.

연산 결과는 즉시 global memory에 write되지 않고 accumulator register fragment에 유지된다.

(2) Epilogue & Visitor Execution

이후 epilogue 단계에서:

```
epilogue(epilogue_visitor, accumulators);
```

형태로 accumulator register가 `EpilogueVisitorLayerNorm`에 전달된다.

Visitor는 accumulator register를 직접 참조하여:

- Bias Add
- Residual Add
- partial statistics computation

을 수행하며, 이후 결과를 global memory에 저장한다.

(3) ThreadblockSwizzle

ThreadblockSwizzle은 CUDA thread block과 matrix tile 간 매핑을 관리한다.

이를 통해 다수의 thread block이 서로 충돌 없이 matrix tile을 병렬 처리할 수 있도록 scheduling을 수행한다.

benchmark_gemm_layernorm.cu

benchmark_gemm_layernorm.cu는 baseline(unfused) 구조와 제안한 fused pipeline의 성능 및 정확도를 비교하기 위한 benchmark 코드이다.

(1) Baseline (Unfused)

Baseline 구현은:

1. GEMM
2. Bias/Residual Add
3. LayerNorm
4. GEMM

을 각각 독립적인 kernel로 수행한다.

이 과정에서 intermediate activation tensor는 각 단계 사이에서 반복적으로 DRAM read/write를 수행한다.

(2) Proposed (Fused)

제안 구조에서는:

- GEMM epilogue fusion
- lightweight reduction kernel
- GEMM mainloop fusion

을 이용하여 3-stage fused pipeline을 구성한다.

이를 통해 intermediate activation tensor의 global memory traffic을 감소시키고 kernel launch overhead를 줄인다.

(3) Numerical Verification

Benchmark 코드에는 fused kernel의 numerical correctness를 검증하기 위한 validation 과정이 포함되어 있다.

- baseline output (`host_v1`)
- fused output (`host_v2`)

간의 차이를 계산하여 numerical error를 측정하며, 추가적으로 NaN 발생 여부를 검사한다.

이를 통해 variance stabilization logic이 정상적으로 동작함을 검증하였다.

E. Additional Notes

1) AI Models and Implementation Details

본 프로젝트에서 AI 모델은 새로운 서비스를 제공하기 위한 기능으로 사용되는 것이 아니라, GPU inference 성능 최적화를 위한 workload로 활용된다. 즉, 본 연구의 목적은 AI 모델을 새롭게 학습시키거나 모델 구조를 변경하는 것이 아니라, GPU에서 이미 학습된 Transformer 기반 모델이 추론될 때 발생하는 kernel-level 병목을 분석하고 이를 최적화하는 것이다.

실험 대상으로는 CLIP, SigLIP-2, BERT, OPT, Llama 계열과 같이 Transformer block을 기반으로 하는 모델 구조를 고려하였다. 이러한 모델들은 세부 구조와 사용 목적은 다르지만, 내부적으로 QKV projection, QKV split 및 transpose, attention 연산, GEMM, residual add, bias add, layer normalization과 같은 공통적인 연산 패턴을 가진다. 본 프로젝트에서는 이 중에서 특히 memory-bound 특성이 강하고 kernel

launch overhead를 유발하는 QKV Split & Transpose와 GEMM 이후의 Epilogue 연산을 주요 최적화 대상으로 선정하였다.

AI 모델의 사용 방식은 다음과 같다. 먼저 기존 AI 모델을 baseline으로 실행하여 Transformer block 내부에서 어떤 CUDA kernel들이 호출되는지 분석한다. 이후 Nsight Systems와 Nsight Compute를 이용해 각 kernel의 실행 시간, DRAM access, arithmetic intensity를 측정한다. 이를 바탕으로 병목이 되는 kernel을 선정하고, 해당 연산을 fused kernel 형태로 재구성한다. 마지막으로 fused kernel을 적용한 결과가 기존 baseline output과 동일하거나 허용 가능한 오차 범위 내에 있는지 검증한다.

따라서 본 프로젝트에서 AI 모델은 학습 대상이 아니라 성능 분석 및 최적화 대상이다. 모델의 weight, 연산 의미, 입력 및 출력 형식은 유지하되, GPU 내부에서 해당 연산이 실행되는 방식을 개선하는 것이 핵심이다. 이러한 접근은 모델 정확도 향상을 목표로 하는 일반적인 AI 연구와 달리, AI 모델의 추론 속도와 실행 효율을 높이는 시스템 최적화 연구에 해당한다.

2) Experimental Setup and Evaluation Metrics

본 프로젝트의 실험은 Transformer 기반 모델의 inference 과정에서 발생하는 kernel 실행 구조를 분석하고, 제안한 kernel fusion 기법의 효과를 평가하는 방식으로 진행된다. 실험은 크게 baseline profiling, fused kernel 구현, correctness 검증, 성능 비교의 단계로 구성된다.

먼저 baseline profiling 단계에서는 기존 PyTorch 또는 TensorRT 기반 모델을 실행하고, Nsight Systems를 이용하여 Transformer block 내부에서 호출되는 CUDA kernel의 순서와 실행 시간을 분석한다. 이를 통해 GEMM, QKV split, transpose, residual add, layer normalization 등 각 연산이 어떤 kernel로 분리되어 실행되는지 확인한다. 이후 Nsight Compute를 사용하여 각 kernel의 DRAM access, memory throughput, arithmetic intensity를 측정함으로써 memory-bound 특성이 강한 kernel을 식별한다.

실험 환경은 서버 GPU 환경과 엡지 GPU 환경을 모두 고려한다. 서버 GPU 환경에서는 fused kernel의 기능적 정확성과 성능 개선 가능성을 우선적으로 검증하고, 이후 Jetson 계열과 같은 엡지 GPU 환경에서 제한된 연산 자원과 메모리 대역폭 조건에서도 동일한 최적화가 효과를 갖는지 확인한다. 특히 엡지 GPU는

데스크탑 GPU에 비해 memory bandwidth와 연산 자원이 제한적이므로, kernel launch overhead와 global memory access를 줄이는 최적화의 효과가 더 크게 나타날 수 있다.

본 프로젝트에서 사용하는 주요 평가 지표는 다음과 같다.

- Inference Latency: baseline 대비 fused kernel 적용 후 전체 추론 시간이 얼마나 감소했는지 측정
- Kernel Launch Count: Transformer block 내부에서 호출되는 CUDA kernel 개수 감소 확인
- Execution Time Ratio: GEMM 대비 비-GEMM kernel이 차지하는 실행 시간 비율 측정
- DRAM Access: 각 kernel에서 발생하는 global memory read/write 양을 측정
- Arithmetic Intensity: 연산량 대비 memory access 비율을 분석하여 memory-bound 여부 판단
- Speedup: baseline 실행 시간 대비 fused implementation의 성능 향상 비율 계산
- Correctness: baseline output과 fused kernel output의 차이를 비교하여 결과 정확성 검증
- Scalability: sequence length, hidden dimension, model type 변화에 따른 최적화 효과 분석

위 항목들은 제안한 Kernel fusion 기법의 효과를 평가하기 위한 기준을 정리한 것이다. 단순히 실행 시간이 감소했는지만 확인한 것이 아니라, Kernel launch 횟수, DRAM access, arithmetic intensity, output correctness를 함께 분석하여 성능 개선의 원인을 구체적으로 파악한다. 예를 들어 QKV Split & Transpose Fusion에서 speedup이 발생한 경우, 그 원인이 split/transpose kernel 제거에 따른 launch overhead 감소인지, intermediate tensor 접근 감소에 따른 DRAM access 감소인지 함께 분석한다.

Correctness 검증 역시 중요한 평가 항목이다. 본 프로젝트는 모델 구조나 연산 의미를 변경하는 것이 아니라 kernel 실행 방식을 최적화하는 것이므로, kernel fusion 적용 이후에도 기존 baseline과 동일한 결과를 출력해야 한다. 이를 위해 baseline output과 fused output을 비교하여 max absolute error, mean absolute error, relative error 등을 측정한다. 특히 GEMM Epilogue Fusion에 포함되는 LayerNorm은 평균과 분산 계산을 포함하는 reduction 연산이므로, 최종 output뿐 아니라 row 단위 mean과 variance 계산이 baseline과 일치하는지도 함께 검증해야 한다.